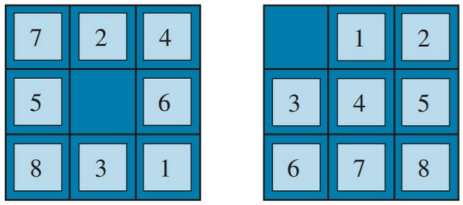
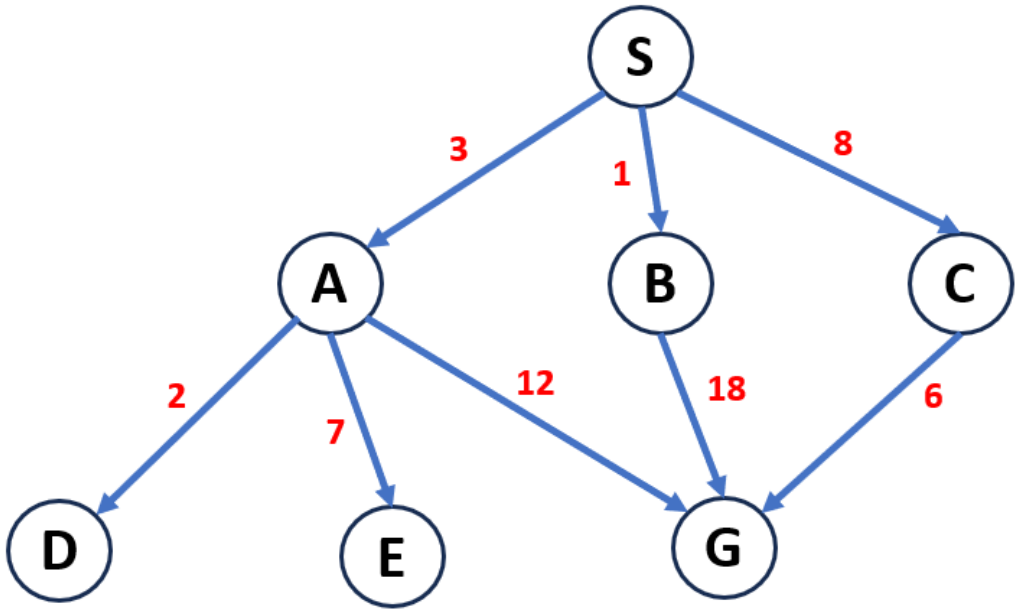
CPSC390 Homework #1

1. **(20 pts)** Using the 8-puzzle in the lecture notes as an example, illustrate what the search tree looks like after expanding the start node. Take one of the child nodes as an example and provide details of the data stored in that specific node data structure.



Start State Goal State

2. For the graph below, the start node is S, and the goal node is G.



Consider **Depth-first Search, Breadth-first Search, and Uniform cost Search, as described by the Best First Search framework:**

Figure 3.7

```
function BEST-FIRST-SEARCH(problem, f) returns a solution node or failure
  node ← NODE(STATE=problem.INITIAL)
  frontier ← a priority queue ordered by f, with node as an element
  reached ← a lookup table, with one entry with key problem.INITIAL and value node
  while not IS-EMPTY(frontier) do
    node ← POP(frontier)
    if problem.IS-GOAL(node.STATE) then return node
    for each child in EXPAND(problem, node) do
      s ← child.STATE
      if s is not in reached or child.PATH-COST < reached[s].PATH-COST then
        reached[s] ← child
        add child to frontier
  return failure

function EXPAND(problem, node) yields nodes
  s ← node.STATE
  for each action in problem.ACTIONS(s) do
    s′ ← problem.RESULT(s, action)
    cost ← node.PATH-COST + problem.ACTION-COST(s, action, s′)
    yield NODE(STATE=s′, PARENT=node, ACTION=action, PATH-COST=cost)
```

Please answer the following questions for each search strategy. Assuming redundant nodes in the frontier are not replaced.

Use the following notation for search nodes:

parent_state->current_state(total_path_cost)

Depth-first Search (20 pts)

(1) Finish the table below to show how the search finds goal node G

Node to be expanded	Frontier
	S(0)
S(0)	S->A(3), S->B(1), S->C(8)

(2) Which Solution path is found?
(3) What is the cost?
(4) What is the number of nodes expanded, i.e., the EXPAND function is called upon?

Breadth-first Search (20 pts)

(1) Finish the table below to show how the search finds goal node G

Node to be Expanded	Frontier
	S(0)
S(0)	S->A(3), S->B(1), S->C(8)

(2) Which Solution path is found?
(3) What is the cost?
(4) What is the number of nodes expanded, i.e., the EXPAND function is called upon?

Uniform cost search: (20 pts)

(1) Finish the table below to show how the search finds goal node G

Node to be expanded	Frontier
	S(0)
S(0)	S->B(1), S->A(3), S->C(8)

(2) Which Solution path is found?
(3) What is the cost?
(4) What is the number of nodes expanded, i.e., the EXPAND function is called upon?

3. Algorithm Analysis

(1) Notice this if condition in the algorithm framework

if s is not in *reached* **or** *child*.PATH-COST < *reached*[s].PATH-COST **then**
  reached[s] ← *child*
  add *child* to *frontier*

a. **(5 pts)** Why is it essential to maintain the *reached* lookup table?

b. **(5 pts)** Is it necessary to do the path cost comparison for all three (BFS, DFS, UCS) algorithms?

(2) **(10 pts)** Notice that the goal check is performed when a node is popped out of the frontier. What will happen if the goal check is performed before a node is added to the frontier instead, i.e., how will the four performance characteristics (completeness, cost optimality, time complexity, and space complexity) change for the three algorithms (BFS, DFS, UCS)?